

Installation Guide

Ticket Management & Version Control Integration System
Group 10 — Microservices Architecture Project

Document	installization.pdf
Version	1.0
Platform	Linux / macOS / Windows (WSL2)
Runtime	Docker 24+ · Docker Compose v2+

1. Prerequisites

Before installing TVCIS, ensure the following tools are installed on your machine:

Tool	Minimum Version	Download URL
Docker Engine	24.0+	https://docs.docker.com/get-docker/
Docker Compose	v2.20+	Bundled with Docker Desktop
Git	2.30+	https://git-scm.com/downloads
curl (optional)	any	For API testing from terminal

■ Verify your Docker installation:

```
docker --version
docker compose version
```

2. Getting the Source Code

Clone the repository from GitHub:

```
git clone https://github.com/group10/ticket-vcs-system.git
cd ticket-vcs-system
```

The project folder structure looks like this:

```
ticket-vcs-system/ user-service/ main.py # FastAPI application
requirements.txt Dockerfile ticket-service/ main.py # Core business logic +
DONE rule requirements.txt Dockerfile vcs-service/ main.py # Git-like commit
simulation requirements.txt Dockerfile docker-compose.yml # Orchestrates all 3
services README.md
```

3. Running the System with Docker Compose

3.1 Build and Start All Services

From the project root directory, run the following command. Docker will build all three service images and start the containers:

```
docker compose up --build
```

On first run, Docker downloads the Python 3.11 base image and installs dependencies (approximately 1–2 minutes). Subsequent starts are much faster.

✓ **Note** — Expected startup output:

```
user-service | INFO: Application startup complete. ticket-service | INFO:
Application startup complete. vcs-service | INFO: Application startup
complete.
```

3.2 Run in Background (Detached Mode)

```
docker compose up --build -d
```

Check that all containers are running:

```
docker compose ps
```

All three services must show Status: healthy. The health checks run every 10 seconds and verify the /health endpoint of each service.

3.3 View Logs

```
# Logs from all services docker compose logs -f
```

```
# Logs from a specific service docker compose logs -f ticket-service
```

3.4 Stop the System

```
docker compose down
```

To also delete all stored data (SQLite volumes):

```
docker compose down -v
```

4. Verifying the Installation

4.1 Health Checks

Open a browser or run the following curl commands to confirm all services are up:

```
curl http://localhost:8001/health # Expected:
{"status":"healthy","service":"user-service"}
```

```
curl http://localhost:8002/health # Expected:
{"status":"healthy","service":"ticket-service"}
```

```
curl http://localhost:8003/health # Expected:
{"status":"healthy","service":"vcs-service"}
```

4.2 Interactive API Documentation

FastAPI automatically generates Swagger UI documentation. Open these URLs in your browser:

- **User Service:** <http://localhost:8001/docs>
- **Ticket Service:** <http://localhost:8002/docs>
- **VCS Service:** <http://localhost:8003/docs>

You can also access ReDoc documentation at /redoc on each port.

5. Complete API Testing Walkthrough

Follow these steps in order to exercise the full Ticket–Commit integration.

Step 1 — Create a Developer User

```
curl -s -X POST http://localhost:8001/users \ -H "Content-Type:
application/json" \ -d '{"username":"alice_dev","email":"alice@example.com",
"full_name":"Alice Smith","role":"developer"}' # Save the returned 'id' field
as USER_ID
```

Step 2 — Create a Ticket

```
curl -s -X POST http://localhost:8002/tickets \ -H "Content-Type: application/json" \ -d '{"title":"Implement dark mode","priority":"HIGH","assignee_id":"","project":"UI"}' # Save the returned 'id' field as TICKET_ID (format: TICK-XXXXXXX)
```

Step 3 — Try to Close the Ticket (should FAIL)

```
curl -s -X PUT http://localhost:8002/tickets/ \ -H "Content-Type: application/json" \ -d '{"status":"DONE"}' # Expected: HTTP 422 with error "DONE_REQUIRES_COMMIT"
```

■ This is expected behaviour — no commit exists yet.

Step 4 — Create a Commit Linked to the Ticket

```
curl -s -X POST http://localhost:8003/commits \ -H "Content-Type: application/json" \ -d '{"message":"feat: add dark mode toggle","ticket_id":"","author_id":"","branch":"feature/dark-mode","repository":"ui-service","files_changed":4,"additions":120,"deletions":5}' # Save the returned 'commit_id' field as COMMIT_ID
```

Step 5 — Verify the Ticket Has Linked Commits

```
curl -s http://localhost:8002/tickets//commits # Expected: {"commit_count": 1, "commits": [...]}
```

Step 6 — Close the Ticket (should SUCCEED)

```
curl -s -X PUT http://localhost:8002/tickets/ \ -H "Content-Type: application/json" \ -d '{"status":"DONE"}' # Expected: HTTP 200 with updated ticket showing status: "DONE"
```

✓ The ticket is now closed. The integration works correctly.

Step 7 — Look Up a Commit's Parent Ticket

```
curl -s http://localhost:8003/commits//ticket # Returns: commit details + the full ticket object it belongs to
```

6. Troubleshooting

Port already in use

- **Cause:** Another process is using port 8001/8002/8003.
- **Fix:** Run: `lsof -i :8001` and kill the process, or edit ports in `docker-compose.yml`.

Container won't start

- **Cause:** Build error or missing dependency.
- **Fix:** Run: `docker compose logs` to inspect error output.

Ticket service returns 503 on DONE

- **Cause:** VCS Service is unreachable at startup.
- **Fix:** Ensure all three services are healthy: `docker compose ps`

Commit rejected (ticket not found)

- **Cause:** The `ticket_id` does not exist in Ticket Service.
- **Fix:** Create the ticket first, then use its exact ID in the commit request.

Data lost after restart

- **Cause:** Volumes were deleted with docker compose down -v.
- **Fix:** Omit the -v flag to preserve data volumes across restarts.

For further support, consult the interactive Swagger UI at /docs on each service port, or refer to the project README.md file in the repository root.